

GUIDE D'IMPLANTATION DES NOTIFICATIONS WEB PUSH — MUZAN SERVICE

React/Vite + Express + PostgreSQL

Objectif : envoyer des notifications système aux utilisateurs et aux administrateurs, même quand l'application est en arrière-plan, avec Web Push et des clés VAPID.

■ ■ **Sécurité** : la clé VAPID privée ne doit jamais être copiée dans un chat, un commit GitHub ou le frontend. Elle doit être ajoutée uniquement dans les secrets du serveur.

1. Installer la dépendance backend

Dans le dossier du backend :

```
pnpm add web-push
pnpm add -D @types/web-push
```

2. Générer et configurer les clés VAPID

Générer une seule paire de clés :

```
node -e "const w=require('web-push'); console.log(w.generateVAPIDKeys())"
```

Ajouter ensuite ces variables dans les secrets du serveur (Replit Secrets, Plesk Environment Variables, etc.) :

```
VAPID_PUBLIC_KEY=<clé publique générée>
VAPID_PRIVATE_KEY=<clé privée générée>
VAPID_CONTACT=mailto:admin@example.com
```

La clé publique peut être envoyée au navigateur. La clé privée reste uniquement sur le backend.

3. Backend : envoyer les notifications

Créer un module similaire à `src/lib/pushNotifications.ts`. La colonne existante **pushToken** contient le JSON de la souscription navigateur.

```
import webpush from "web-push";

let initialized = false;

function initPush() {
  if (initialized) return;
  const publicKey = process.env.VAPID_PUBLIC_KEY;
  const privateKey = process.env.VAPID_PRIVATE_KEY;
  if (!publicKey || !privateKey) return;

  webpush.setVapidDetails(
    process.env.VAPID_CONTACT ?? "mailto:admin@example.com",
    publicKey,
    privateKey
  );
  initialized = true;
}

export interface PushMessage {
  title: string;
  body: string;
  data?: Record<string, unknown>;
}

function parseSubscription(token: string | null | undefined) {
  if (!token) return null;
  try {
    const value = JSON.parse(token);
    return value?.endpoint ? value : null;
  } catch {
    return null;
  }
}

export async function sendPushNotification(
  tokens: (string | null | undefined)[],
  message: PushMessage
) {
  await initPush();
  for (const token of tokens) {
    if (!token) continue;
    const subscription = parseSubscription(token);
    if (!subscription) continue;
    webpush.sendNotification(subscription, message, {
      vapid: {
        publicKey,
        privateKey,
        email: process.env.VAPID_CONTACT ?? "mailto:admin@example.com"
      }
    });
  }
}
```

```

initPush();
if (!initialized) return;

const payload = JSON.stringify(message);
await Promise.allSettled(
  tokens
    .map(parseSubscription)
    .filter(Boolean)
    .map((subscription) =>
      webpush.sendNotification(subscription!, payload)
    )
);
}

export async function broadcastPushNotification(
  tokens: (string | null | undefined)[],
  message: PushMessage
) {
  return sendPushNotification(tokens, message);
}

```

4. Backend : routes API

Ajouter une route publique pour la clé publique et une route protégée pour enregistrer la souscription :

```

router.get("/config/push/vapid-public-key", (_req, res) => {
  const publicKey = process.env.VAPID_PUBLIC_KEY;
  if (!publicKey) {
    res.status(503).json({ error: "Push non configuré" });
    return;
  }
  res.json({ publicKey });
});

router.post("/push-token", requireAuth, async (req, res) => {
  const { token } = req.body;
  if (typeof token !== "string" || !token) {
    res.status(400).json({ error: "token requis" });
    return;
  }
  await db.update(usersTable)
    .set({ pushToken: token })
    .where(eq(usersTable.id, req.userId!));
  res.json({ success: true });
});

```

Lors d'un événement (nouveau dépôt, retrait, VIP ou message admin), appeler :

```

await sendPushNotification([user.pushToken], {
  title: "Dépôt confirmé",
  body: "Votre dépôt a été confirmé.",
  data: { url: "/transactions" }
});

await broadcastPushNotification(allUserTokens, {
  title: "Message de l'administration",
  body: "Votre message ici.",
  data: { url: "/notifications" }
});

```

5. Service Worker : afficher la notification

Dans **public/sw.js**, conserver le cache PWA existant et ajouter :

```

self.addEventListener("push", (event) => {
  let data = {
    title: "Nouvelle notification",
    body: "Vous avez une nouvelle notification."
  };
  try {
    if (event.data) data = event.data.json();
  } catch {}

  event.waitUntil(
    self.registration.showNotification(data.title, {
      body: data.body,
      icon: "/icon-192.png",
      badge: "/icon-192.png",
      data: data.data ?? {},
      vibrate: [200, 100, 200]
    })
  );
});

```

```
self.addEventListener("notificationclick", (event) => {
  event.notification.close();
  const url = event.notification.data?.url ?? "/";
  event.waitUntil(clients.openWindow(url));
});
```

6. Frontend : créer la souscription

Créer `src/lib/pushSubscription.ts` :

```
function keyToBytes(key: string) {
  const padding = "=".repeat((4 - key.length % 4) % 4);
  const base64 = (key + padding).replace(/-/g, "+").replace(/_/g, "/");
  return Uint8Array.from(atob(base64), c => c.charCodeAt(0));
}

export async function subscribeToPush() {
  if (!("serviceWorker" in navigator) || !("PushManager" in window)) return;
  if (Notification.permission === "denied") return;

  const keyResponse = await fetch("/api/config/push/vapid-public-key");
  if (!keyResponse.ok) return;
  const { publicKey } = await keyResponse.json();
  const registration = await navigator.serviceWorker.ready;

  let subscription = await registration.pushManager.getSubscription();
  if (!subscription) {
    if (Notification.permission === "default") {
      if (await Notification.requestPermission() !== "granted") return;
    }
    subscription = await registration.pushManager.subscribe({
      userVisibleOnly: true,
      applicationServerKey: keyToBytes(publicKey)
    });
  }

  const token = localStorage.getItem("auth_token");
  await fetch("/api/push-token", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      ...(token ? { Authorization: `Bearer ${token}` } : {})
    },
    body: JSON.stringify({ token: JSON.stringify(subscription) })
  });
}
```

7. Appeler après la connexion

```
import { subscribeToPush } from "@lib/pushSubscription";

// Après login réussi, ou au chargement d'une session déjà connectée :
setTimeout(() => subscribeToPush().catch(() => {}), 1500);
```

8. Vérification avant déploiement

- Vérifier que le site est en HTTPS (obligatoire hors localhost).
- Vérifier que **sw.js** est accessible à la racine du domaine.
- Connecter un utilisateur et accepter la permission de notification.
- Vérifier que le champ **pushToken** est rempli en base.
- Déclencher une notification depuis l'admin et vérifier l'affichage sur téléphone.
- Redémarrer le backend après avoir ajouté les variables VAPID.

9. Fonctionnement global

Le navigateur enregistre le Service Worker → le frontend demande la permission → une souscription Push est créée avec la clé publique VAPID → la souscription est envoyée au backend → le backend la stocke dans `pushToken` → lors d'un événement, Express utilise web-push pour envoyer la notification → le Service Worker reçoit l'événement push et affiche la notification système.